

Part 3: Sql challenge

Answers:

1. The strategy of use indexing in the database.

An index in a database is similar to an index in a book. It allows the database to quickly locate data without scanning the entire table. Without indexes, queries often require full table scans, which are slow for large datasets.

- Primary purpose: Improve query performance.
- Trade-off: Indexes consume storage and slow down write operations (INSERT, UPDATE, DELETE).

Types of Indexes

1. **Primary Key Index**
 - Automatically created for primary keys.
 - Ensures uniqueness.
2. **Unique Index**
 - Ensures that values in the column(s) are unique.
3. **Single-column Index**
 - Index on a single column to speed up searches.
4. **Composite (Multi-column) Index**
 - Indexes on multiple columns.
 - Useful when queries filter or sort on multiple fields.
5. **Full-text Index**
 - For searching text fields efficiently.
6. **Clustered vs Non-clustered Index**
 - Clustered: Rows are stored physically in the order of the index.
 - Non-clustered: Separate structure pointing to rows.

Indexing Strategy

A good indexing strategy requires balancing performance gains vs storage and write cost. Here's how to approach it:

A. Identify Query Patterns

- Analyze which queries are most frequent.
- Focus on columns used in:
 - WHERE clauses
 - JOIN conditions
 - ORDER BY or GROUP BY

Example: If you frequently query `SELECT * FROM orders WHERE customer_id = ?`, indexing `customer_id` makes sense.

B. Use Selective Indexes

- Selectivity: Ratio of unique values to total rows.
- High selectivity → good candidate for an index.

- Low selectivity (e.g., gender column with only M or F) → index may not help much.

C. Composite Indexes

- Useful when queries filter on multiple columns.
- Important: Column order matters!
 - INDEX(a, b) is useful for queries filtering by a alone or a and b.
 - Not useful if filtering only by b.

D. Avoid Over-Indexing

- Every index adds storage cost.
- Write-heavy tables (lots of INSERT/UPDATE/DELETE) suffer performance hits.
- Rule of thumb: Index only what queries need.

E. Monitor and Adjust

- Use EXPLAIN or query planner tools to see which indexes are used.
- Remove unused or redundant indexes.
- Periodically rebuild indexes to avoid fragmentation.

F. Special Cases

- Foreign keys: Often indexed to speed up joins.
- Sorting: Indexes can speed up ORDER BY.
- Partial or filtered indexes: Index only rows meeting certain criteria.
-

2. The top performers –

If I have a table Students with the following columns:

StudentID (integer)
 Name (varchar(100))
 Subject (varchar(100))
 Score (integer)

Using mssql && using this query give below , I can found the top scorer in each subject :

```
SELECT StudentID, Name, Subject, Score
FROM (
  SELECT
    StudentID,
    Name,
    Subject,
    Score,
    RANK() OVER (PARTITION BY Subject ORDER BY Score DESC) AS rnk
  FROM Students
) t
WHERE rnk = 1;
```

Screenshot:

SQLQuery2.sql - DESKTOP-98KLLIL\sajjad (60)

```

SELECT TOP (1000) [StudentID],[Name],[Subject],[Score] FROM [Test_crud].[dbo].[Students]

SELECT StudentID, Name, Subject, Score
FROM (
    SELECT
        StudentID,
        Name,
        Subject,
        Score,
        RANK() OVER (PARTITION BY Subject ORDER BY Score DESC) AS rnk
    FROM Students
) t
WHERE rnk = 1;

```

StudentID	Name	Subject	Score
2	Bob	Math	85
3	Charlie	Math	90
4	David	Science	88
5	Eva	Science	92
6	Frank	Science	80
7	Grace	English	91
8	Helen	English	89

Query executed successfully. DESKTOP-98KLLIL\SQLEXPRESS ... DESKTOP-98KLLIL\sajjad... Test_crud 00:00:00 12 rows

3. Answers :

3.1 : Calculating the total quantity_sold from the Sales table :

Screenshot:

SQLQuery6.sql - DESKTOP-98KLLIL\sajjad (53)

```

SELECT TOP (1000) [sale_id],
[product_id],
[quantity_sold],
[sale_date],
[total_price]
FROM [Test_crud].[dbo].[Sales]

SELECT SUM(quantity_sold) AS Total_Quantity_Sold
FROM Sales;

```

sale_id	product_id	quantity_sold	sale_date	total_price
1	101	5	2024-01-01	2500.00
2	102	3	2024-01-02	900.00
3	103	2	2024-01-02	60.00
4	104	4	2024-01-03	80.00
5	105	6	2024-01-03	90.00

Total_Quantity_Sold
20

Query executed successfully. DESKTOP-98KLLIL\SQLEXPRESS ... DESKTOP-98KLLIL\sajjad... Test_crud 00:00:00 6 rows

3.2 : Retrieving product_name and unit_price from the Products table with the Highest Unit Price

Screenshot:

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure, including tables, views, and security. The SQL Query window on the right contains the following query:

```
SELECT TOP (1000) [product_id]
,[product_name]
,[category]
,[unit_price]
FROM [Test_crud].[dbo].[Products]

SELECT product_name, unit_price
FROM Products
WHERE unit_price = (
SELECT MAX(unit_price)
FROM Products
);
```

The Results pane shows the output of the query, displaying a list of products with their IDs, names, categories, and unit prices. The product with the highest unit price (500.00) is highlighted.

product_id	product_name	category	unit_price
101	Laptop	Electronics	500.00
102	Smartphone	Electronics	300.00
103	Headphones	Electronics	30.00
104	Keyboard	Electronics	20.00
105	Mouse	Electronics	15.00

product_name	unit_price
Laptop	500.00

3.3 : Finding the Products Not Sold from Products table.

Screenshot:

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure, including tables, views, and security. The SQL Query window on the right contains the following query:

```
SELECT * FROM Products;
SELECT * FROM Sales;

SELECT p.product_id, p.product_name
FROM Products p
LEFT JOIN Sales s
ON p.product_id = s.product_id
WHERE s.product_id IS NULL;
```

The Results pane shows the output of the query, displaying a list of products with their IDs, names, categories, and unit prices. The product with the highest unit price (500.00) is highlighted.

product_id	product_name	category	unit_price
101	Laptop	Electronics	500.00
102	Smartphone	Electronics	300.00
103	Headphones	Electronics	30.00
104	Keyboard	Electronics	20.00
105	Mouse	Electronics	15.00
106	Light	Electronics	5.00

sale_id	product_id	quantity_sold	sale_date	total_price
1	101	5	2024-01-01	2500.00
2	102	3	2024-01-02	900.00
3	103	2	2024-01-02	60.00
4	104	4	2024-01-03	80.00
5	105	6	2024-01-03	90.00

product_id	product_name
106	Light

Query executed successfully. DESKTOP-98KLLIL\SQL EXPRESS ... DESKTOP-98KLLIL\sajjad... Test_crud 00:00:00 12 rows